Polishers Da Elite Cleaning Services: Website and Admin Dashboard Guide

Design and implementation plan for the public website and back office. Stack: React, Vite, Supabase, Vercel, Git. Version 1.0, November 2026.

1. Overview

Goal. A fast, trustworthy website that turns visitors into cleaning requests, plus a private admin dashboard where your team can (a) receive and manage those requests and (b) update the landing page content (hero, services, gallery, testimonials, promotions) without touching code.

Brand line: *Dirt doesn't stand a chance!* The site should feel premium, clean and confident: lots of white space, crisp photography, one strong call to action everywhere: **Request a Quote**.

Two experiences, one codebase

Area	Desktop	Mobile
Public site	Classic website: top navigation, wide sections, multi-column grids	App-style screens: bottom tab bar, large cards, bottom sheets, sticky call-to-action
Admin dashboard	Sidebar + data tables + side panels	Bottom tabs + card lists + full-screen detail pages

What the site does

- Presents the four service lines: residential, commercial, industrial and post-construction cleaning (plus estates and offices).
- Collects quote and booking requests through a short step-by-step form.
- Stores every request in Supabase and notifies the team instantly.
- Lets admins manage request status, notes and assignments.
- Lets admins upload and edit landing page content, which goes live immediately.

2. Design System

The palette below is a suggestion that fits the premium, clean feel. Swap in your logo colours if you already have them.

Token	Value	Use
`--navy`	`#0B1F3A`	Headings, nav, footer, dark sections
`--aqua`	`#14B8C4`	Primary buttons, links, active states
`--gold`	`#C9A227`	Small accents (badges, stars, the Elite touch)
`--mist`	`#F4F8FB`	Section backgrounds
`--ink`	`#1F2937`	Body text
`--success` / `--warn` / `--danger`	`#16A34A` / `#F59E0B` / `#DC2626`	Status colours in admin

Typography. Headings: Poppins or Plus Jakarta Sans (600/700). Body: Inter (400/500). Mobile body size 16px minimum.

Shape and motion. 16px rounded cards on mobile, 12px on desktop, soft shadows, 200ms transitions. A subtle sparkle or shine animation on the hero heading reinforces the *polished* idea without being childish.

Responsive breakpoints. Mobile under 768px, tablet 768 to 1024px, desktop above 1024px. Design mobile first, then enhance upward.

Core components (build once, reuse everywhere): `Button`, `Card`, `Badge`, `Input`, `Select`, `Stepper`, `BottomSheet`, `Modal`, `Toast`, `DataTable`, `EmptyState`, `Skeleton`, `ImageUploader`.

3. Architecture

```
Browser (React + Vite SPA)
   |  supabase-js (anon key)
   v
Supabase
   - Auth (admin login)
   - Postgres (requests, content tables) + Row Level Security
   - Storage (images in the site-media bucket)
   - Realtime (live new-request alerts)
   - Edge Function (email / WhatsApp notification, spam check)
   ^
Vercel hosts the built site; every Git push redeploys.
```

Why this works. Supabase gives you the database, login, file storage and live updates without writing a separate server. Row Level Security (RLS) is what keeps the public safe from admin data, so it must be configured carefully (section 9).

Suggested libraries

- `react-router-dom` for routing
- `@supabase/supabase-js` for backend access
- `@tanstack/react-query` for data fetching and caching
- `react-hook-form` + `zod` for forms and validation
- `framer-motion` for transitions
- `tailwindcss` for styling (or CSS modules if you prefer)
- `lucide-react` for icons

Folder structure

```
src/
  lib/supabase.js
  hooks/            useServices, useRequests, useAuth ...
  components/ui/    Button, Card, BottomSheet ...
  layouts/          PublicLayout, AdminLayout
  pages/
    public/         Home, Services, ServiceDetail, Gallery, About, Contact,
Quote
    admin/          Login, Dashboard, Requests, RequestDetail, Content/*,
Media, Team, Settings
  routes/           ProtectedRoute.jsx
  styles/
```

Environment variables (Vercel project settings and a local `.env`, never committed):

```
VITE_SUPABASE_URL=...
VITE_SUPABASE_ANON_KEY=...
```

The anon key is safe in the browser only because RLS protects the data. Never put the `service_role` key in the frontend.

4. Sitemap and Roles

Public routes: `/` Home, `/services`, `/services/:slug`, `/gallery`, `/about`, `/contact`, `/quote` (request form), `/quote/success`.

Admin routes: `/admin/login`, `/admin` (dashboard), `/admin/requests`, `/admin/requests/:id`, `/admin/content/hero`, `/admin/content/services`, `/admin/content/gallery`, `/admin/content/testimonials`, `/admin/content/faqs`, `/admin/content/promotions`, `/admin/media`, `/admin/team`, `/admin/settings`.

Roles

Role	Can do
`super_admin`	Everything, including adding or removing team members and settings
`admin`	Manage requests and all landing page content
`staff`	View and update requests assigned to them; no content or team access

5. Public Website: Screens and Flows

5.1 Home page

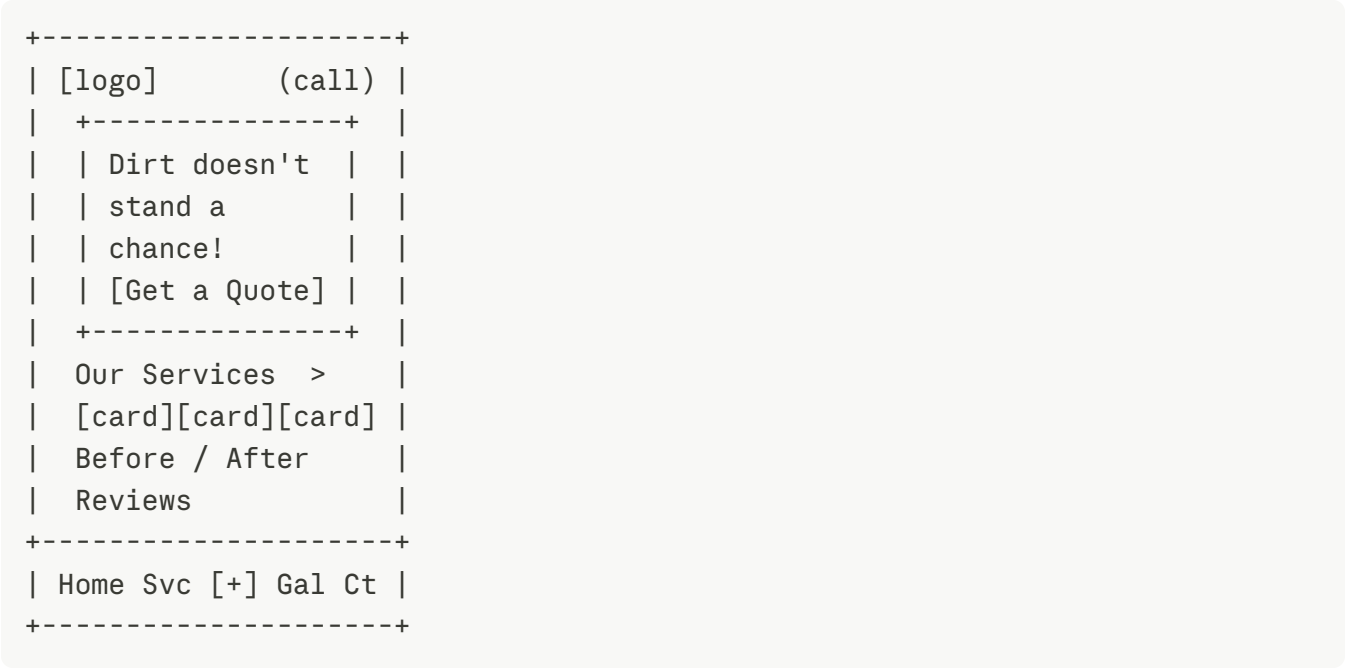
Desktop layout (top to bottom)

1. **Header:** logo left; links Services, Gallery, About, Contact; gold-outlined **Request a Quote** button right.
2. **Hero:** full-width image or slider (content comes from the admin), headline, the line *Dirt doesn't stand a chance!*, two buttons: **Get a Free Quote** and **WhatsApp Us**.
3. **Trust bar:** counters such as jobs completed, happy clients, years of experience (editable in admin).
4. **Services grid:** four to six cards, each linking to its detail page.
5. **How it works:** three steps: Request, We Confirm and Inspect, We Clean and You Relax.
6. **Before and after gallery:** slider comparing results.
7. **Why choose us:** the POLISHERS values (Professionalism, Ownership, Leadership, Integrity, Service, High standards, Excellence, Reliability, Satisfaction) as a compact icon grid.
8. **Testimonials carousel.**
9. **Promotions banner** (appears only when an active promotion exists).
10. **FAQ accordion.**
11. **Final call to action** and **footer** (contacts, socials, service areas).

Mobile (app-style)

- A compact top bar with the logo and a phone icon replaces the full header.

- A **bottom tab bar** with five items: Home, Services, Gallery, Quote (raised centre button), Contact.
- Hero becomes a rounded card with the headline and one big button.
- Services become a **horizontally swipeable card row**; gallery becomes a swipe carousel.
- A floating WhatsApp button sits above the tab bar.
- Sections use generous padding, 16px rounded cards and subtle shadows so it feels like a native app screen rather than a shrunken website.



Implementation. `Home.jsx` fetches published content with React Query (`site_content`, `services`, `gallery_items`, `testimonials`, `faqs`, `promotions`). Each section component takes data as props and renders skeletons while loading. Use one `useIsMobile()` hook (matchMedia at 768px) only where layouts truly differ; use CSS breakpoints everywhere else. `PublicLayout` renders the top header on desktop and `BottomTabBar` on mobile.

5.2 Services and service detail

Flow: Home or Services card, then service detail, then **Request a Quote** with the service pre-selected.

- **Services page:** grid of cards (image, title, one-line description, *Learn more*). Mobile uses a vertical list of large image cards.

- **Detail page:** hero image, full description, what is included (checklist), who it is for, typical process, related gallery photos, FAQ, sticky **Request a Quote** bar on mobile.
- **Implementation:** route `/services/:slug`; query `services` by `slug` where `is_published = true`; the quote button navigates to `/quote?service=<slug>`.

5.3 Gallery

Filter chips (All, Residential, Commercial, Industrial, Post-construction). Desktop uses a masonry grid with a lightbox; mobile uses a two-column grid, tapping opens a full-screen viewer with swipe. Before and after items show a draggable comparison slider. Images come from Supabase Storage; use `loading="lazy"` and Supabase image transformations (width parameter) for small thumbnails.

5.4 About and Contact

- **About:** story, mission, the nine POLISHERS values, team photos, certifications if any.
- **Contact:** phone, WhatsApp, email, address, service areas, opening hours, and a short message form (saved as a request with type `enquiry`). Tap-to-call and tap-to-WhatsApp buttons on mobile.

5.5 Quote request flow (the most important public flow)

Keep it short. Every extra field costs conversions, especially on mobile.

Step	Fields
1. Service	Choose service type (cards): Residential, Commercial, Industrial, Post-construction, Other
2. Property	Property type, approximate size or number of rooms, location or area
3. Schedule	Preferred date, preferred time (Morning / Afternoon / Flexible)
4. Contact	Full name, phone (WhatsApp-friendly), email (optional), extra notes, optional photo upload
5. Review and submit	Summary card with edit links, then **Submit Request**

Desktop: a centred card with a progress stepper on top and a summary panel on the right. **Mobile:** one question group per full screen, a progress bar at the top, a large **Continue** button pinned to the bottom, and the keyboard-friendly input types (`tel`, `email`, native date picker).

After submit: the success screen shows the reference number (for example `PDE-000123`), a promise such as *We will contact you within 24 hours*, and buttons for **Chat on WhatsApp** and **Back to Home**.

Implementation

```
// lib/supabase.js
import { createClient } from '@supabase/supabase-js';
export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY
);
```

```
// submitting a request (anon users may INSERT only)
const { data, error } = await supabase
  .from('service_requests')
  .insert({
    full_name, phone, email, service_type, property_type,
    size_note, address, preferred_date, preferred_time, message,
  })
  .select('reference')
  .single();
```

Note: anonymous users cannot read rows back, so the `.select('reference')` above only works if you return the reference through a small Edge Function or an RPC (`submit_request`) that inserts and returns the reference. Use the RPC approach; it also lets you add spam checks. Validate with `zod` on the client and again in the database with `check` constraints. Add a hidden honeypot field and Cloudflare Turnstile (free) to block bots.

6. Admin Dashboard (Back Office)

This is the control centre. Design it so a non-technical person can run the whole website from a phone.

6.1 Login and access

Flow: `/admin/login`, then enter email and password, then Supabase Auth session, then redirect to `/admin`. Unauthorised users are bounced to login. Offer *Forgot password* (Supabase email reset). There is no public sign-up: a super admin invites team members.

UI. Desktop: split screen, brand image on the left, form on the right. Mobile: logo on top, form card, big **Sign In** button.

Implementation

```
// routes/ProtectedRoute.jsx
import { Navigate } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth';

export default function ProtectedRoute({ children, roles }) {
  const { session, profile, loading } = useAuth();
  if (loading) return <FullScreenSpinner />;
  if (!session) return <Navigate to='/admin/login' replace />;
  if (roles && !roles.includes(profile?.role)) return <Navigate to='/admin'
replace />;
  return children;
}
```

`useAuth` subscribes to `supabase.auth.onAuthStateChange`, then loads the matching row from `profiles` to know the role. Remember that this guard is only a convenience; real protection is RLS in the database.

6.2 Admin layout

Desktop: left sidebar (Dashboard, Requests with a red unread badge, Content, Media, Team, Settings), top bar with search, notification bell and user menu, main content area.

Mobile (app style): top bar with page title and bell; **bottom tab bar** with Home, Requests, Content, Media, More. Lists become cards, filters become a bottom sheet, detail pages open full screen with a back arrow, and primary actions (such as *Add new*) use a floating action button.

```
DESKTOP                                    MOBILE
+------+------------------------------+    +---------------------+
| Logo | Search        (bell) (user) |    | Requests     (bell) |
| Dash +------------------------------+    | [New][Open][Done]   |
| Reqs |  KPI  KPI  KPI  KPI          |    | +-----------------+ |
| Cont |  Chart      Recent requests  |    | | PDE-000123      | |
| Media|                              |    | | Chika O.  [New] | |
| Team |                              |    | | Post-constr.    | |
| Sett |                              |    | +-----------------+ |
+------+------------------------------+    | Home Reqs Cont ...  |
                                           +---------------------+
```

Implementation. `AdminLayout` wraps all `/admin/*` routes inside `ProtectedRoute`. It renders `Sidebar` above 1024px and `AdminBottomTabs` below. Use `<Outlet />` for page content.

6.3 Dashboard (overview)

Purpose: answer *what needs my attention today?* in five seconds.

- **KPI cards:** New requests (unread), Requests this week, Jobs scheduled, Conversion rate (completed vs received).
- **Recent requests:** last 8, with status badges, tap to open.
- **Requests over time:** a simple line or bar chart for 30 days.
- **Requests by service:** a small donut or bar chart.
- **Quick actions:** Add promotion, Upload to gallery, Edit hero.

Mobile: KPI cards in a horizontal swipe row or 2x2 grid, then recent requests as a card list, charts below.

Implementation. One Postgres view or RPC (`dashboard_stats`) returns all counts in a single call, so the dashboard loads quickly. Charts via `recharts`. Subscribe to Realtime inserts so the **New requests** counter increments live and a toast appears.

```
supabase
  .channel('requests-live')
  .on('postgres_changes',
      { event: 'INSERT', schema: 'public', table: 'service_requests' },
      (payload) => {
        queryClient.invalidateQueries({ queryKey: ['requests'] });
        toast(`New request from ${payload.new.full_name}`);
      })
  .subscribe();
```

6.4 Requests inbox

This is the daily-use screen.

Request lifecycle (status pipeline)

`New` then `Contacted` then `Quoted` then `Scheduled` then `In Progress` then `Completed`, with `Cancelled` and `Lost` as exits.

Status	Meaning	Badge colour
New	Just received, not seen	Aqua
Contacted	Team has reached the client	Blue
Quoted	Price sent	Amber
Scheduled	Date confirmed	Purple
In Progress	Team on site	Orange
Completed	Job done	Green
Cancelled / Lost	Did not proceed	Grey / Red

Desktop UI: a table with columns Reference, Client, Service, Area, Preferred date, Status, Assigned to, Received. Above it: search box, status tabs with counts, filters for service, date range and assignee, and an Export CSV button. Clicking a row opens the detail view (either a right-hand slide-over panel or a full page).

Mobile UI: a vertical list of request cards (client name, service, status badge, time ago, a green tap-to-call icon). Filter chips scroll horizontally under the title; *More filters* opens a bottom sheet. Swipe a card to reveal quick actions (Call, WhatsApp, Mark contacted). Unread requests have a bold title and a coloured dot.

Flow: open inbox, filter by *New*, tap a request, read details, call or WhatsApp the client, change status to *Contacted*, add a note. The status change and note are saved to history.

Implementation

```
// useRequests.js
const { data, error, count } = await supabase
  .from('service_requests')
  .select('*, assignee:profiles(full_name)', { count: 'exact' })
  .eq('status', status)               // optional filter
  .ilike('full_name', `%${search}%`) // optional search
  .order('created_at', { ascending: false })
  .range(from, to);                  // pagination, 20 per page
```

Use React Query with `keepPreviousData` for smooth paging, and server-side pagination and filtering so the page stays fast with thousands of rows. CSV export builds the file in the browser from the current filtered result.

6.5 Request detail

Sections

1. **Header:** reference, status dropdown, assignee dropdown, received time.
2. **Client card:** name, phone (tap to call), WhatsApp button, email, address, map link.
3. **Job details:** service, property type, size, preferred date and time, client message, uploaded photos.
4. **Quote:** optional amount and notes (visible only to admins).
5. **Internal notes:** chat-style thread, newest first, with author and time.
6. **Activity timeline:** automatic log such as *Status changed from New to Contacted by Ada*.

Mobile: full-screen page. A sticky bottom bar holds **Call** and **WhatsApp**, and the status control is a bottom sheet with large tappable options.

Implementation. When an admin opens a `New` request, update it to `Contacted` only on explicit action, but set `seen_at` automatically so the unread badge clears. Status changes go through one function so history is always recorded:

```
async function updateStatus(id, status) {
  await supabase.from('service_requests').update({ status }).eq('id', id);
  // a database trigger writes to request_activity (who, when, from, to)
}
```

WhatsApp link format: `https://wa.me/<phone-with-country-code>?text=<encoded message>`. Normalise phone numbers to international format (for Nigeria, convert a leading 0 to 234) when saving.

6.6 Content management (landing page editor)

The rule: **anything a visitor sees on the landing page should be editable here.** Each content type has the same pattern: list, add or edit form, publish toggle, reorder, and live preview.

Module	What admins manage
Hero	Headline, subtext, button labels, background image or slides, on/off
Services	Title, slug, short and full description, icon, image, included items, display order, published
Gallery	Upload photos, before and after pairs, category, caption, order, published
Testimonials	Client name, role, quote, rating, photo, published
FAQs	Question, answer, order, published
Promotions	Title, description, banner image, start and end date, button link, active
Stats	Jobs completed, clients, years (the trust bar numbers)
Contact and socials	Phone, WhatsApp, email, address, hours, social links

Desktop UI: left tab list of modules; right side is a table or card grid. **Add or Edit** opens a slide-over panel with a form on the left and a **live preview** of how it will look on the site on the right. **Drag handles** reorder items.

Mobile UI: module cards on the Content tab; tapping one opens a list with thumbnails; **+** floating button opens a full-screen form; reordering uses move up and down buttons instead of drag. A **Published** switch at the top of every form.

Flow (example: upload a gallery photo): Content, Gallery, **+ Add**, pick photo from camera or gallery, choose category, add caption, optionally add the *before* photo, toggle Published, Save. The photo appears on the live site immediately after saving.

Implementation. Public pages read only rows where `is_published = true`; admin screens read everything. Drafts are simply unpublished rows. For the many small single-value texts (hero headline, stats, contact info), keep one `site_content` table of key and JSON value pairs so adding a new editable text never needs a migration.

```
// reorder after drag
await supabase.from('services').upsert(
  items.map((item, index) => ({ id: item.id, sort_order: index }))
);
```

After any save, call `queryClient.invalidateQueries` in the admin. On the public site, React Query refetches on page load, so visitors see changes within seconds with no redeploy. Zod schemas validate the forms (title required, slug lowercase and unique, image required for gallery).

6.7 Media library and image upload

UI. A grid of uploaded images with search, copy-link and delete. Desktop supports drag-and-drop; mobile opens the camera or photo library. Show upload progress and a thumbnail instantly.

Implementation

```
async function uploadImage(file, folder = 'gallery') {
  const ext = file.name.split('.').pop();
  const path = `${folder}/${crypto.randomUUID()}.${ext}`;
  const { error } = await supabase.storage
    .from('site-media')
    .upload(path, file, { cacheControl: '31536000', upsert: false });
  if (error) throw error;
  return supabase.storage.from('site-
media').getPublicUrl(path).data.publicUrl;
}
```

Compress images in the browser before upload (the `browser-image-compression` library) to about 1600px wide so phone photos do not slow the site. Allow only JPG, PNG, WebP and cap size at 5MB. Deleting content should also delete its file, or you slowly accumulate unused images.

6.8 Team and settings

- **Team (super admin only):** list of users with role and active toggle; **Invite** sends an email invite through an Edge Function (it needs the service role key, so it must never run in the browser). Change role or deactivate a user.
- **Settings:** business name, logo, notification email and WhatsApp number, service areas, working hours, request auto-reply message, maintenance mode toggle.

6.9 Notifications

When a request is submitted, three things happen: the dashboard updates live (Realtime), the team receives an email, and optionally a WhatsApp or SMS alert goes out. Implement with a **Database Webhook** on `service_requests` INSERT that calls a Supabase **Edge Function**, which sends the email through Resend (or Brevo) and can call a WhatsApp or SMS provider. The client also receives an automatic confirmation email if they gave one. On mobile, installing the site as a **PWA** (add to home screen) gives admins an app-like icon; push notifications can be added later.

7. Database Schema

Run this in the Supabase SQL editor. It is a solid starting point; adjust fields to your needs.

```
-- roles
create type user_role as enum ('super_admin','admin','staff');
create type request_status as enum

('new','contacted','quoted','scheduled','in_progress','completed','cancelled'
,'lost');

create table profiles (
  id uuid primary key references auth.users on delete cascade,
  full_name text,
  role user_role not null default 'staff',
  is_active boolean not null default true,
  created_at timestamptz default now()
```

```
);

create sequence request_ref_seq start 1;

create table service_requests (
  id uuid primary key default gen_random_uuid(),
  reference text unique default ('PDE-' ||
lpad(nextval('request_ref_seq')::text, 6, '0')),
  full_name text not null check (char_length(full_name) between 2 and 120),
  phone text not null,
  email text,
  service_type text not null,
  property_type text,
  size_note text,
  address text,
  preferred_date date,
  preferred_time text,
  message text check (char_length(message) <= 2000),
  photo_urls text[] default '{}',
  status request_status not null default 'new',
  assigned_to uuid references profiles(id),
  quote_amount numeric,
  seen_at timestamptz,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

create table request_notes (
  id uuid primary key default gen_random_uuid(),
  request_id uuid references service_requests(id) on delete cascade,
  author_id uuid references profiles(id),
  body text not null,
  created_at timestamptz default now()
);

create table request_activity (
  id uuid primary key default gen_random_uuid(),
  request_id uuid references service_requests(id) on delete cascade,
  actor_id uuid references profiles(id),
  action text not null,        -- e.g. 'status_changed'
  from_value text, to_value text,
  created_at timestamptz default now()
);
```

```
create table site_content (key text primary key, value jsonb not null,
updated_at timestamptz default now());

create table services (
  id uuid primary key default gen_random_uuid(),
  slug text unique not null, title text not null,
  short_desc text, description text, icon text, image_url text,
  includes text[] default '{}',
  sort_order int default 0, is_published boolean default true
);

create table gallery_items (
  id uuid primary key default gen_random_uuid(),
  title text, category text, image_url text not null, before_url text,
  sort_order int default 0, is_published boolean default true,
  created_at timestamptz default now()
);

create table testimonials (
  id uuid primary key default gen_random_uuid(),
  name text not null, role text, quote text not null,
  rating int check (rating between 1 and 5), avatar_url text,
  sort_order int default 0, is_published boolean default true
);

create table faqs (
  id uuid primary key default gen_random_uuid(),
  question text not null, answer text not null,
  sort_order int default 0, is_published boolean default true
);

create table promotions (
  id uuid primary key default gen_random_uuid(),
  title text not null, description text, image_url text, link text,
  starts_at timestamptz, ends_at timestamptz, is_active boolean default true
);
```

8. Security: Row Level Security and Storage

Turn on RLS for every table, then add policies. Helper functions keep policies short:

```
create function is_admin() returns boolean language sql security definer
stable as $$
  select exists (select 1 from profiles
    where id = auth.uid() and is_active and role in ('admin','super_admin'));
$$;

create function is_staff_or_admin() returns boolean language sql security
definer stable as $$
  select exists (select 1 from profiles where id = auth.uid() and is_active);
$$;

alter table service_requests enable row level security;
alter table services enable row level security;
-- repeat for every table

-- visitors may submit requests but never read them
create policy 'public can submit' on service_requests
  for insert to anon with check (status = 'new' and assigned_to is null);

-- admins see and edit all; staff see only their assigned requests
create policy 'admin all' on service_requests for all to authenticated
  using (is_admin()) with check (is_admin());
create policy 'staff assigned' on service_requests for select to
authenticated
  using (assigned_to = auth.uid());

-- public content: read published only, admins write
create policy 'public read' on services for select to anon, authenticated
  using (is_published);
create policy 'admin write' on services for all to authenticated
  using (is_admin()) with check (is_admin());
```

Create the same pair of policies for `gallery_items`, `testimonials`, `faqs`, `promotions` and `site_content` (public read, admin write). Note: Postgres policy names are identifiers, so in the actual editor use double quotes around the policy names, such as `"public can submit"`.

Storage. Create a bucket `site-media` marked public for reading. Add storage policies so only admins can insert, update or delete objects in that bucket. If clients can attach photos to requests, use a separate bucket `request-uploads` that allows anonymous upload (size and type limited) but admin-only reading through signed URLs.

Other protections

- Add Cloudflare Turnstile and a honeypot field on the quote form, and rate limit through an Edge Function.
- Never expose the `service_role` key in the frontend or commit it to Git.
- Add a trigger that keeps `updated_at` current and another that logs status changes into `request_activity`.
- Enable email confirmation and a strong password policy in Supabase Auth; consider enabling MFA for super admins.
- Test RLS by calling the API with the anon key and confirming it cannot read requests.

9. SEO, Performance and PWA

- Use `react-helmet-async` for page titles and descriptions; add Open Graph images so links look good on WhatsApp.
- Add LocalBusiness structured data (JSON-LD) with name, services and service area.
- A Vite SPA is client-rendered, so keep the first screen light, lazy load routes with `React.lazy`, and serve optimised WebP images. If SEO becomes critical later, pre-render public pages or migrate them to a framework such as Next.js.
- Add `vite-plugin-pwa` for a manifest and icons so both the public site and the admin can be installed to the phone home screen, which strengthens the app-like feel.
- Add `vercel.json` with a rewrite so deep links such as `/admin/requests` work on refresh:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

10. Git, Environments and Deployment

Branching. `main` is production; work on `feature/...` branches and open pull requests. Vercel automatically builds a **preview URL** for each branch so you can test on a phone before merging.

Environments. Create two Supabase projects: `polishers-dev` and `polishers-prod`. Use the dev keys in Vercel preview and local `.env`, and prod keys in Vercel production. Keep your SQL in a `supabase/migrations` folder in Git (Supabase CLI) so both databases stay identical.

Deploy steps

1. Push the repo to GitHub.
2. Import the repo in Vercel (framework preset: Vite, build `npm run build`, output `dist`).
3. Add `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` in Vercel settings.
4. Run the migrations and policies on the prod Supabase project.
5. Create the first super admin: sign up the user in Supabase Auth, then insert a `profiles` row with role `super_admin`.
6. Connect your custom domain in Vercel and set the Supabase Auth Site URL and redirect URLs to that domain.

11. Build Roadmap

Phase	Deliverables	Est. time
1. Foundation	Vite project, design tokens, UI components, layouts, routing, Supabase setup and migrations	3 to 4 days
2. Public site	Home, Services, Detail, Gallery, About, Contact with mobile tab bar	5 to 7 days
3. Quote flow	Multi-step form, validation, RPC, success screen, spam protection	3 to 4 days
4. Admin core	Auth, admin layout, dashboard, requests inbox and detail, notes, statuses	6 to 8 days
5. Content manager	Hero, services, gallery, testimonials, FAQs, promotions, media library	6 to 8 days
6. Polish and launch	Notifications, PWA, SEO, RLS testing, performance, deploy, training the team	4 to 5 days

Times assume one developer working steadily; adjust for your pace.

12. Launch Checklist

- RLS enabled on every table and tested with the anon key.
- First super admin created and a second admin added as backup.

- Test request submitted from a real phone; notification email and WhatsApp link both work.
- Real photos, testimonials and contact details loaded through the admin.
- Lighthouse scores checked on mobile (aim for 90 or above on performance and accessibility).
- Custom domain, HTTPS and Supabase redirect URLs confirmed.
- Database backups enabled in Supabase; admin guide shared with the team.

13. Suggested Next Steps

The most useful next move is to start with Phase 1 and the database schema, since everything else depends on them. I can also produce a clickable visual mockup of the home page and the admin dashboard on desktop and mobile, or scaffold the actual Vite project with the routing, layouts and Supabase hooks ready to run.